CS-BAPR: Causal-Symbolic Bayesian Adaptive Policy Regularization with Machine-Verified OOD Extrapolation Bounds*

Yifan Zhang† Liang Zheng‡

Abstract

Standard reinforcement learning policies trained on bounded state regions fail catastrophically when deployed in out-of-distribution (OOD) regimes. A key yet underexplored source of this failure is *architectural*: ReLU-based policy networks exhibit derivative discontinuities that permit unbounded error growth in extrapolation regions—a structural deficiency that no amount of data can remedy.

We propose **CS-BAPR** (Causal-Symbolic Bayesian Adaptive Policy Regularization), which extends the BAPR framework [1] with three pillars for principled OOD extrapolation: (1) *Neural Arithmetic Units* (NAU/NMU) that replace ReLU output layers, providing architecturally guaranteed finite derivative Lipschitz constants; (2) *SINDy symbolic discovery* that extracts sparse analytic equations from trajectory data and enforces Jacobian consistency between the policy gradient and the discovered physical law; and (3) *IRM causal filtering* that selects symbolic formulas invariant across multiple training environments, provably tightening extrapolation bounds.

Our main theoretical result is the **OOD Extrapolation Stability Bound**: if the policy achieves base accuracy δ on the training domain, SINDy derivative error ε, and NAU/NMU derivative Lipschitz constant L, then the OOD prediction error at distance $\|d\|$ satisfies $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. We further prove that ReLU networks *provably lack* any finite L, establishing a sharp architectural boundary for extrapolation capability. The CS-BAPR Bellman operator with frozen symbolic penalty Γ_{sym} is a γ-contraction, extending BAPR's convergence guarantee without additional assumptions. All results—including the ReLU impossibility theorem, the full SINDy-to-OOD pipeline, and the n-dimensional quadratic bound—are machine-verified in Lean 4 with no `sorry` (954 lines, 34 theorems). A dedicated counterproof (`CSBAPR-Counterproof.lean`) establishes that the frozen-Γ_{sym} requirement is *necessary*.

Keywords: out-of-distribution extrapolation; symbolic regression; neural arithmetic units; causal invariance; formal verification; robust reinforcement learning.

1 Introduction

Reinforcement learning (RL) has achieved remarkable success in domains where training and deployment conditions are well-aligned [3]. However, real-world applications frequently require policies to operate in regimes far beyond their training distribution—a challenge we term *magnitude extrapolation*. Unlike the interpolation-based generalization studied in domain randomization [5] or the regime-switch adaptation addressed by BAPR [1], magnitude extrapolation demands that the agent's output scale correctly to input magnitudes never encountered during training.

Definition 1.1 (Extrapolation Failure)**.** *Let $\pi_\theta : \mathcal{S} \to \mathcal{A}$ be a policy trained on domain $D_{train} \subset \mathcal{S}$, and let $f_{real} : \mathcal{S} \to \mathcal{A}$ be the optimal response function governed by underlying physical laws. The policy suffers from* extrapolation failure *if there exists a distance threshold $d_0 > 0$ such that for x_{ood} with $\|x_{ood} - D_{train}\| > d_0$:*

*Code available at https://github.com/erzhu419/CS-BAPR

†Central South University, erzhu419@gmail.com, 204201048@csu.edu.cn

‡Central South University, zhengliang@csu.edu.cn

(i) ***Error explosion:*** $\|\pi_\theta(x_{ood}) - f_{real}(x_{ood})\|$ *grows super-polynomially with distance;*

(ii) ***Gradient saturation:*** $\|\nabla_x \pi_\theta(x_{ood})\| \to 0$ *or exhibits discontinuous jumps, preventing meaningful response scaling.*

This failure mode is *architecturally determined*—it arises not from insufficient data or poor training, but from the structural properties of the function class used to represent the policy. We identify three root causes and their corresponding solutions:

1. **ReLU derivative discontinuity (architectural):** Standard ReLU networks have piecewise-linear activations whose derivatives are discontinuous at breakpoints. In OOD regions, the derivative can change arbitrarily between 0 and 1, causing the policy's response to "freeze" or "jump" unpredictably. We prove (machine-verified) that no finite Lipschitz constant L exists for ReLU derivatives, making polynomial OOD error bounds *of the form* $\delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ structurally impossible for ReLU policies (Theorem 4.12). Note that this does not preclude other framework-specific OOD guarantees for ReLU networks; rather, it establishes that the derivative-Lipschitz approach—which yields the tightest bounds in our framework—is architecturally incompatible with ReLU.

2. **Lack of physical structure (representational):** Even with smooth activations, a purely data-driven policy has no mechanism to discover that the environment obeys sparse analytic laws (e.g., $F = ma$, ballistic equations). Without such structure, the policy must memorize input-output mappings rather than learning generalizable rules. We address this through SINDy-based symbolic discovery [14], which extracts sparse differential equations from trajectory data and conditions the policy's gradient to align with the discovered law (Jacobian Consistency Loss).

3. **Spurious correlations (statistical):** A formula discovered from a single training environment may capture correlations (e.g., temperature $\leftrightarrow$ wind speed) rather than causal mechanisms (e.g., wind $\to$ trajectory deviation). We integrate Invariant Risk Minimization (IRM) [17] to filter symbolic candidates across multiple environments, retaining only causally invariant physical laws that remain stable under distributional shift.

Building upon the robust ensemble framework of RE-SAC [2] and the Bayesian change-point adaptation of BAPR [1], our contributions are:

- **OOD Extrapolation Stability Bound (Theorem 4.35):** Under Jacobian Consistency (ε) and NAU/NMU Arithmetic Bias ($L < \infty$), the OOD error satisfies $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. To our knowledge, this is the first machine-verified polynomial OOD bound for structured RL policies satisfying the Jacobian Consistency and Arithmetic Bias conditions.

- **ReLU Impossibility Theorem (Theorem 4.12):** $\neg\exists L$, $\text{HasArithmeticInductiveBias}(\text{ReLU}, L)$. Machine-verified in Lean 4, establishing that ReLU networks are structurally incapable of satisfying any polynomial OOD bound of this form.

- **SINDy-to-OOD Pipeline (Theorem 4.18):** A formal composition: SINDy identification success $\Rightarrow$ Jacobian Consistency $\Rightarrow$ OOD Bound, providing end-to-end guarantees from data-driven symbolic discovery to extrapolation quality.

- **IRM Tightening (Theorem 4.20):** IRM-filtered symbolic formulas yield provably tighter Jacobian Consistency bounds than single-environment formulas, reducing the ε term in the OOD bound.

- **CS-BAPR Bellman Contraction (Theorem 4.23):** The CS-BAPR operator—extending BAPR with a frozen symbolic consistency penalty Γ_{sym}—is a γ-contraction in L_∞, preserving convergence guarantees. The symbolic penalty cancels exactly in the contraction argument, requiring only that it be frozen during each backup.

- **Machine verification:** All results are formally verified in Lean 4 with no `sorry` (`CSBAPR.lean`: 954 lines, 34 definitions/theorems/lemmas), using Mathlib's calculus library including the convex Mean Value Theorem and the Grönwall-style fencing theorem.

2 Related work

2.1 OOD generalization in RL

Generalization in RL has been studied through domain randomization [5], distributional robustness [9, 8], and epistemic uncertainty quantification [10]. These approaches provide *interpolation* guarantees within the convex hull of training environments but do not address *extrapolation* to magnitudes beyond the training support. Robust MDPs [6, 7] hedge against worst-case perturbations but with *static* uncertainty sets that become vacuous far from the training distribution. CS-BAPR instead leverages *structural* knowledge (physical laws) to extend guarantees beyond the training domain.

2.2 Neural arithmetic units

Neural Arithmetic Logic Units (NALU) [11] introduced the idea of discrete-weight arithmetic layers for numerical extrapolation. Neural Arithmetic Units (NAU) and Neural Multiplication Units (NMU) [12] improved stability by separating addition and multiplication paths and using regularization-based weight discretization. These architectures have been applied to function learning tasks [13] but not to RL policy networks or formal OOD bounds. CS-BAPR is the first to provide machine-verified Lipschitz derivative guarantees for NAU/NMU architectures and to prove that ReLU lacks this property.

2.3 Symbolic regression and SINDy

Sparse Identification of Nonlinear Dynamics (SINDy) [14] discovers governing equations from data using sparse regression over a candidate function library. SINDy-RL [15] integrates SINDy with model-based RL for interpretable dynamics models. AI Feynman [16] uses physics-inspired symbolic regression for equation discovery. CS-BAPR formalizes the SINDy identification process in Lean 4 and proves that successful identification implies Jacobian Consistency, which in turn implies polynomial OOD bounds.

2.4 Causal RL and invariant risk minimization

Invariant Risk Minimization (IRM) [17] seeks predictors whose optimal classifier is invariant across training environments, targeting causal rather than spurious correlations. Causal representation learning [18] and causal RL surveys [19] provide the theoretical foundations. CS-BAPR applies IRM not to the policy directly but to the *symbolic formula selection* stage, filtering SINDy candidates to retain only causally invariant physical laws.

2.5 Physics-informed ML and ReLU failures

Recent work has identified systematic failures of ReLU activations in physics-informed machine learning [20], showing that ReLU's piecewise linearity prevents accurate learning of smooth physical solutions. Our ReLU Impossibility Theorem (Theorem 4.12) provides the first *formal proof* that this failure extends to derivative Lipschitz continuity—a necessary condition for polynomial OOD bounds within our framework. We emphasize that the impossibility is specific to bounds requiring Lipschitz-continuous derivatives; alternative frameworks (e.g., piecewise-linear extrapolation analysis) may yield different guarantees for ReLU networks.

2.6 Formal verification in RL

Machine verification of RL algorithms remains rare. Our prior work established Lean 4 contraction proofs for RE-SAC [2] and BAPR [1]. CS-BAPR extends this methodology to OOD extrapolation bounds, SINDy formalization, IRM consistency, and architectural impossibility results—the broadest scope of machine-verified RL theory to date.

3 Preliminaries

3.1 Maximum entropy RL

Following SAC [3], the agent maximizes the entropy-augmented objective:

$$J(\pi) = \mathbb{E}_{\pi,P}\left[\sum_{t=0}^{\infty}\gamma^t\left(R(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))\right)\right], \tag{1}$$

where $\mathcal{H}(\pi(\cdot|s_t))$ is the Shannon entropy and α is the temperature parameter.

3.2 BAPR framework

CS-BAPR builds upon the BAPR framework [1], which integrates Bayesian Online Change Detection (BOCD) [4] with robust ensemble RL for piecewise-stationary environments. The BAPR operator is a belief-weighted mixture of mode-conditional Bellman operators:

$$\mathcal{T}_\rho^{BAPR}Q(s,a) = \sum_{m\in\mathcal{M}}\rho(m)\cdot\mathcal{T}_m Q(s,a), \tag{2}$$

where ρ is a frozen belief distribution over modes derived from the BOCD posterior. Each mode-conditional operator $\mathcal{T}_m$ inherits RE-SAC's disentangled aleatoric (κ) and epistemic (Γ_{epi}) penalties [2]. The operator is a γ-contraction when all penalties and beliefs are frozen during each backup [1].

4 Methodology

4.1 The OOD extrapolation problem

Let $D_{\text{train}} \subset \mathcal{S}$ be the training domain and $x_0 \in D_{\text{train}}$ a reference point. For an OOD test point $x_{\text{ood}} \notin D_{\text{train}}$, we seek to bound $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\|$ as a function of the distance $d = x_{\text{ood}} - x_0$.

Remark 4.1 (On the relationship between π and f_{real})**.** *In standard RL, the policy $\pi : \mathcal{S} \to \mathcal{A}$ maps states to actions, while the environment dynamics $f_{real} : \mathcal{S}\times\mathcal{A} \to \mathcal{S}$ maps state-action pairs to next states. These are semantically distinct. Our OOD bound applies in the important special case where the policy approximates a* deterministic optimal response function *$\pi^* : \mathcal{S} \to \mathcal{A}$ governed by known physical laws (e.g., ballistic corrections, torque control). In this setting, $f_{real}(s)$ denotes the* physics-prescribed optimal action *at state s, and the Jacobian Consistency constraint $\nabla_s\pi \approx \nabla_s f_{sym}$ ensures that the policy's* rate of change *tracks the physical law's rate of change in OOD regions. This is meaningful because: (i) in control problems with analytic solutions, the optimal policy* is *a known function of state; (ii) SINDy discovers equations governing how the* optimal response *changes with state, not the transition dynamics per se; and (iii) the gradient alignment ensures that even when the policy has not seen a particular state magnitude, its response scales consistently with the underlying physics.*

We state four structural assumptions under which the CS-BAPR framework operates:

Assumption 4.2 (SINDy Identifiability)**.** *The optimal response function $f_{real} : \mathcal{S} \to \mathcal{A}$ admits a sparse representation in the chosen basis library $\{\varphi_1, \ldots, \varphi_n\}$, such that SINDy can identify it with derivative accuracy $\varepsilon_1 < \varepsilon_{threshold}$ on the training domain.*

Assumption 4.3 (Smooth Target Dynamics)**.** *The function f_{real} is differentiable on the convex hull of $D_{train} \cup \{x_{ood}\}$. This is satisfied for physical systems governed by smooth differential equations (e.g., rigid-body dynamics, fluid mechanics, ballistic trajectories), but excludes environments with discontinuous transitions (e.g., contact-rich manipulation with Coulomb friction).*

Assumption 4.4 (Bounded Feature Extractor)**.** *The feature extraction layers of the actor network have a finite Lipschitz constant $L_{feat} < \infty$, achievable via spectral normalization or weight clamping. The effective derivative Lipschitz constant of the composed actor is $L_{eff} = L_{feat} \cdot L_{head}$, where L_{head} is the NAU/NMU output head's Lipschitz derivative constant (0 for NAU, $2|c|$ for NMU).*

Assumption 4.5 (Pointwise Jacobian Consistency on OOD Segment)**.** *The trained policy satisfies ε-Jacobian Consistency* pointwise *along the line segment from x_0 to x_{ood}, i.e., $\forall t \in [0,1]$, $\|D\pi(x_0 + t \cdot d) - Df_{sym}(x_0 + t \cdot d)\| \leq \varepsilon_2$. In practice, the Jacobian Consistency Loss $\mathcal{L}_{AC}$ (Eq. (7)) is enforced in expectation over mini-batches, not pointwise. This assumption is justified by: (i) the smoothness of neural networks with Lipschitz-bounded weights ensures that pointwise bounds on sampled points extend to nearby regions via continuity; (ii) the NAU/NMU output head has* exactly *constant or polynomial derivatives, so the Jacobian Consistency achieved at training points holds globally for the output head; and (iii) the SINDy formula f_{sym} is an analytic function, so $\nabla_s f_{sym}$ is smooth and slowly-varying, meaning training-domain alignment propagates to nearby OOD regions. Standard uniform convergence arguments [26] guarantee that $\varepsilon_{population} \leq \varepsilon_{empirical} + O(1/\sqrt{N})$ for the Jacobian loss evaluated on N i.i.d. samples.*

We introduce two key predicates, formalized in Lean 4:

Definition 4.6 (Jacobian Consistency)**.** *A function $\pi : E \to F$ is ε-*Jacobian Consistent *with a symbolic formula $f_{sym} : E \to F$ on domain D if:*

$$\forall x \in D, \quad \|D\pi(x) - Df_{sym}(x)\| \leq \varepsilon, \tag{3}$$

where D denotes the Fréchet derivative. (Lean: `IsJacobianConsistent_nD`, line 400.)

Definition 4.7 (Arithmetic Inductive Bias)**.** *A function $\pi : E \to F$ has L-*Arithmetic Inductive Bias *if its Fréchet derivative is L-Lipschitz:*

$$\forall x_1, x_2, \quad \|D\pi(x_1) - D\pi(x_2)\| \leq L \cdot \|x_1 - x_2\|. \tag{4}$$

(Lean: `HasArithmeticInductiveBias_nD`, line 404.)

4.2 Pillar I: NAU/NMU arithmetic bias

The first pillar replaces the policy's output layers with Neural Arithmetic Units, ensuring finite derivative Lipschitz constants.

NAU layer. A Neural Arithmetic Unit [12] computes $y = \sum_j w_j x_j$ with weights constrained to $\{-1, 0, 1\}$ (realized via clamping and regularization). Since the output is linear in the input, the derivative $\partial y / \partial x_i = w_i$ is *constant*—independent of the input value.

Theorem 4.8 (NAU has zero-Lipschitz derivatives)**.** *For any integer weight $w \in \mathbb{Z}$, the function $f(x) = wx$ satisfies $HasArithmeticInductiveBias(f, 0)$. (Lean: `nau_has_arithmetic_bias_1d`, line 843.)*

Proof. $f'(x) = w$ for all x, so $|f'(x_1) - f'(x_2)| = |w - w| = 0 \leq 0 \cdot |x_1 - x_2|$. □

For multi-dimensional inputs, the NAU derivative constancy extends naturally:

Theorem 4.9 (NAU partial derivatives are constant)**.** *For an NAU layer with integer weights $w_1, \ldots, w_n \in \mathbb{Z}$ computing $y = \sum_j w_j x_j$, the partial derivative $\partial y / \partial x_i = w_i$ is* constant *everywhere—independent of the input vector. This guarantees that the derivative does not drift in OOD regions. (Lean: `nau_derivative_is_constant`, line 817.)*

NMU layer. A Neural Multiplication Unit [11] computes $f(x) = c \cdot x^2$ with learnable coefficient c. Its derivative $f'(x) = 2cx$ is Lipschitz with constant $L = 2|c|$.

Theorem 4.10 (NMU has bounded-Lipschitz derivatives)**.** *For any coefficient $c \in \mathbb{R}$, HasArithmeticInductiveBias$(x \mapsto cx^2, 2|c|)$. (Lean: `nmu_has_bounded_arithmetic_bias`, line 876.)*

Proof. $f'(x) = 2cx$, so $|f'(x_1) - f'(x_2)| = |2cx_1 - 2cx_2| = 2|c| \cdot |x_1 - x_2|$. □

Corollary 4.11 (NAU ∘ NMU composition)**.** *For weights $w, c \in \mathbb{R}$, HasArithmeticInductiveBias$(x \mapsto wc \cdot x^2, 2|wc|)$. (Lean: `nau_nmu_composition_bias`, line 896.)*

ReLU impossibility. In contrast, we prove that ReLU networks cannot satisfy any polynomial OOD bound:

Theorem 4.12 (ReLU derivative is not Lipschitz)**.** $\neg\exists L \in \mathbb{R}$, *HasArithmeticInductiveBias$(ReLU, L)$. (Lean: `relu_derivative_not_lipschitz`, line 908.)*

Proof sketch. $\mathrm{ReLU}(x) = \max(0, x)$ has derivative 0 for $x < 0$ and 1 for $x > 0$. For any purported L:

- If $L \leq 0$: take $x_1 = -1, x_2 = 1$. Then $|0 - 1| = 1 \leq L \cdot 2 \leq 0$, contradiction.
- If $L > 0$: take $x_1 = -(4L)^{-1}, x_2 = (4L)^{-1}$. Then $|0-1| = 1 \leq L \cdot 2(4L)^{-1} = 1/2$, contradiction.

The full proof is machine-verified in Lean 4 using filter-based derivative computation. □

Remark 4.13 (Architectural hierarchy for extrapolation)**.** *The machine-verified results establish a sharp hierarchy:*

Architecture	L	***OOD bound***	***Lean theorem***
NAU $(w \cdot x)$	0	δ *(constant)*	`nau_has_arithmetic_bias_1d`
NMU $(c \cdot x^2)$	$2\lvert c\rvert$	$\delta + \varepsilon\lVert d\rVert + \lvert c\rvert\lVert d\rVert^2$	`nmu_has_bounded_arithmetic_bias`
ReLU $(\max(0, x))$	$\nexists$	*unbounded*	`relu_derivative_not_lipschitz`

Remark 4.14 (Theory-practice gap: composed architecture)**.** *The Lean 4 proofs establish Lipschitz derivative constants for* isolated *NAU ($L = 0$) and NMU ($L = 2|c|$) layers. The actual actor is a composition $\pi = \tanh \circ (\alpha \cdot NAU + (1-\alpha) \cdot NMU) \circ FeatureNet$. The composed constant satisfies $L_{eff} \leq L_{feat} \cdot 2(1-\alpha)|c| \cdot 1 < \infty$ (Assumption 4.4), preserving the quadratic OOD bound. The key structural insight: NAU/NMU output heads yield finite L, while ReLU output heads provably do not. See Appendix F for the full L_{eff} derivation.*

4.3 Pillar II: SINDy symbolic discovery

The second pillar uses Sparse Identification of Nonlinear Dynamics (SINDy) [14] to extract analytic equations from trajectory data, then enforces alignment between the policy's gradient and the discovered law.

Basis library. We define a library of n differentiable candidate functions $\varphi_1, \ldots, \varphi_n : E \to F$ (e.g., $\{1, x, x^2, \sin x, e^x\}$) where each φ_i is differentiable on the domain. The symbolic formula space is the span $\{f_{\text{sym}} = \sum_{i=1}^{n} \xi_i \varphi_i \mid \xi \in \mathbb{R}^n\}$. (Lean: `BasisLibrary`, line 456.)

SINDy identification. Given trajectory data, SINDy solves:

$$\xi = \arg\min_{\xi} \|\dot{X} - \Theta(X)\xi\|_2 + \lambda\|\xi\|_1, \tag{5}$$

where $\Theta(X)$ is the basis function matrix evaluated on data. The reconstructed formula is $f_{\text{sym}}(x) = \sum_i \xi_i \varphi_i(x)$.

We formalize successful identification as a predicate on output quality:

Definition 4.15 (SINDy Identification Success)**.** *SINDy identifies the true dynamics f_{real} with derivative accuracy ε_1 on domain D if there exist sparse coefficients ξ such that:*

$$\forall x \in D, \quad \|Df_{real}(x) - D(\textstyle\sum_i \xi_i \varphi_i)(x)\| \leq \varepsilon_1. \tag{6}$$

(Lean: `SINDyIdentifies`, line 502.)

Theorem 4.16 (SINDy $\Rightarrow$ Jacobian Consistency)**.** *If SINDy identifies f_{real} with derivative error ε_1, and the policy π is trained to match the SINDy formula with derivative error ε_2, then π is $(\varepsilon_1+\varepsilon_2)$-Jacobian Consistent with f_{real}. (Lean: `sindy_implies_jacobian_consistency_nD`, line 518.)*

Proof sketch. Triangle inequality: $\|D\pi - Df_{\text{real}}\| \leq \|D\pi - Df_{\text{sym}}\| + \|Df_{\text{sym}} - Df_{\text{real}}\| \leq \varepsilon_2 + \varepsilon_1$. □

Corollary 4.17 (Tight version with same coefficients)**.** *When the policy is trained against the* same *SINDy formula that was identified, i.e., both SINDy error and training error are measured against the same coefficients ξ, the Jacobian Consistency follows directly with the same $\varepsilon_1+\varepsilon_2$ bound. (Lean: `sindy_implies_jacobian_consistency_tight`, line 546.) This is the version actually used in the pipeline proof (Theorem 4.18).*

Analytic Consistency Loss. In training, the Jacobian Consistency is enforced through a gradient-alignment loss:

$$\mathcal{L}_{\text{AC}} = \mathbb{E}_{s\sim\mathcal{D}}\left[\|\nabla_s \pi_\theta(s) - \nabla_s f_{\text{sym}}(s)\|^2\right], \tag{7}$$

where f_{sym} is the frozen SINDy formula (its gradients are detached to satisfy the frozen-parameter requirement).

Theorem 4.18 (Full SINDy-to-OOD Pipeline)**.** *If SINDy identifies f_{real} with derivative error ε_1 globally, the policy is trained with derivative error ε_2 globally, $\|\pi(x_0) - f_{real}(x_0)\| \leq \delta$, and $\varepsilon_1+\varepsilon_2 \geq 0$, then:*

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \leq \delta + (\varepsilon_1+\varepsilon_2)\|x_{ood} - x_0\|. \tag{8}$$

(Lean: `sindy_to_ood_bound_nD`, line 576.) This linear bound is the special case of the main quadratic result (Theorem 4.35) when $L = 0$ (pure NAU architecture). When NMU layers are used ($L = 2|c|$), the quadratic term $(L/2)\|d\|^2$ appears.

Proof sketch. By Theorem 4.16, π is $(\varepsilon_1+\varepsilon_2)$-Jacobian Consistent with f_{real}. Setting $C = \varepsilon_1+\varepsilon_2$ in Theorem 4.33 yields the result immediately. □

4.4 Pillar III: IRM causal filtering

The third pillar filters symbolic formula candidates using Invariant Risk Minimization [17] to retain only causally invariant physical laws. A SINDy formula fitted from a single environment may capture *spurious correlations*—statistical associations (e.g., temperature $\leftrightarrow$ air density in ballistic control) that hold in the training environment but break under distribution shift. IRM addresses this by requiring the symbolic formula to maintain accuracy across *multiple* environments with distinct dynamics parameters but shared causal structure.

Definition 4.19 (IRM Consistency)**.** *A SINDy formula with coefficients ξ is ε-*IRM Consistent *across environments $\{e_1, \ldots, e_K\}$ if:*

$$\forall e \in \{e_1, \ldots, e_K\},\ \forall x \in D_e, \quad \|Df^e_{real}(x) - Df_{sym}(x)\| \leq \varepsilon. \tag{9}$$

(Lean: `IRM_Consistent`, line 726.)

Theorem 4.20 (IRM Tightening)**.** *If formula ξ_{irm} is ε_{irm}-IRM Consistent and formula ξ_{raw} achieves $\varepsilon_{raw} \geq \varepsilon_{irm}$ in a single environment, then the IRM-filtered formula yields Jacobian Consistency bound $\varepsilon_{irm} + \varepsilon_{pol} \leq \varepsilon_{raw} + \varepsilon_{pol}$. (Lean: `irm_tightens_jacobian_consistency`, line 736.)*

Proof. Direct from $\varepsilon_{\text{irm}} \leq \varepsilon_{\text{raw}}$: $\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}} \leq \varepsilon_{\text{raw}} + \varepsilon_{\text{pol}}$. □

Theorem 4.21 (Full IRM-to-OOD Pipeline)**.** *If formula ξ_{irm} is ε_{irm}-IRM Consistent, the policy is trained with derivative error ε_{pol} against the IRM formula globally, and $\|\pi(x_0) - f_{real}(x_0)\| \leq \delta$ in environment e, then:*

$$\|\pi(x_{ood}) - f^e_{real}(x_{ood})\| \leq \delta + (\varepsilon_{irm} + \varepsilon_{pol})\|x_{ood} - x_0\|. \tag{10}$$

This is the IRM analogue of the SINDy pipeline (Theorem 4.18), composing IRM filtering with the n-D OOD bound. (Lean: `irm_to_ood_bound_nD`, line 755.)

IRM filtering protocol. IRM filtering is applied during Phase 0 (pre-training) and requires $K \geq 3$ environments with distinct dynamics parameters (e.g., different gravity levels, friction coefficients, or traffic demand patterns) but shared causal structure:

1. Collect exploration trajectories from each environment e_k;
2. Fit SINDy independently on each environment's data;
3. Evaluate each candidate formula's derivative error ε_k across *all* environments;
4. Select the coefficient set with the smallest cross-environment derivative variance: $\xi_{\text{irm}} = \arg\min_\xi \text{Var}_k[\varepsilon_k(\xi)]$.

The selected formula is frozen for the entirety of Phase 1 training.

Remark 4.22 (Environment diversity requirement)**.** *IRM filtering depends critically on environment diversity. With too few environments ($K < 3$) or insufficiently diverse dynamics, IRM cannot distinguish causal from spurious features [17]. In practice, environments should vary in parameters that* do not *affect the target physical law (e.g., varying friction but holding gravity constant when the target law is ballistic). If only a single environment is available, CS-BAPR degrades gracefully: the IRM step is skipped and the single-environment SINDy formula is used directly, with ε_{raw} replacing ε_{irm} in the OOD bound.*

4.5 CS-BAPR Bellman operator

CS-BAPR extends the BAPR operator [1] with a frozen symbolic consistency penalty Γ_{sym}. The mode-conditional operator is:

$$\mathcal{T}_h^{CS} Q(s,a) = R_h(s,a) + \gamma \left(\sum_{s'} P_h(s'|s,a) V^Q(s') - \lambda_{\text{epi}} \Gamma^h_{\text{epi}}(s,a) - \lambda_{\text{sym}} \Gamma^h_{\text{sym}}(s,a) - \kappa_{\text{ale}} \right), \tag{11}$$

where $\Gamma^h_{\text{sym}}(s,a) = \|\nabla_s \pi(s) - \nabla_s f_{\text{sym}}(s)\|^2$ captures the Jacobian deviation between the policy and the SINDy formula. The CS-BAPR operator is the belief-weighted mixture:

$$\mathcal{T}_\rho^{CS\text{-}BAPR} Q(s,a) = \sum_{h \in \mathcal{H}} \rho(h) \cdot \mathcal{T}_h^{CS} Q(s,a). \tag{12}$$

Theorem 4.23 (CS-BAPR Contraction)**.** *Under frozen belief $\rho \geq 0$ with $\sum_h \rho(h) = 1$, per-mode stochastic transitions, and frozen penalties (κ_{ale}, Γ_{epi}, Γ_{sym}), the operator $\mathcal{T}_\rho^{CS\text{-}BAPR}$ is a γ-contraction in L_∞ with a unique fixed point. (Lean: `csbapr_contraction`, line 299.)*

Proof sketch. The proof exploits that all three penalty terms (Γ_{epi}, Γ_{sym}, κ_{ale}) are frozen during each backup and therefore cancel in the contraction argument:

$$\begin{aligned} &|\mathcal{T}_h^{CS} Q_1(s,a) - \mathcal{T}_h^{CS} Q_2(s,a)| \\ &= \gamma \left| \sum_{s'} P_h(s'|s,a)(V^{Q_1}(s') - V^{Q_2}(s')) \right| \leq \gamma\varepsilon. \end{aligned} \tag{13}$$

The mixture over modes preserves contraction since ρ is a probability distribution. The full proof, including Blackwell's sufficiency conditions, is machine-verified. □

Corollary 4.24 (Contraction after belief update)**.** *If the belief ρ is updated via Bayesian normalization $\rho'(h) = \rho(h) \cdot L(h)/Z$ with non-negative likelihoods and positive normalizer, then $\mathcal{T}_{\rho'}^{CS\text{-}BAPR}$ remains a γ-contraction. (Lean: `csbapr_contraction_after_update`, line 370.)*

Theorem 4.25 (Necessity of frozen Γ_{sym}: contraction breaks under Q-dependent symbolic penalty)**.** *Consider the CS-BAPR operator on a single-state single-action MDP where Γ_{sym} depends on Q with coupling sensitivity $\mu \geq 0$, defined as $\mu = |\partial\Gamma_{sym}/\partial Q|$ evaluated at the relevant stationary point (i.e., $\Gamma_{sym}(Q) \approx C - \mu Q$ due to online SINDy refinement creating a feedback loop: higher $Q \to$ better policy $\to$ lower Jacobian error $\to$ lower Γ_{sym}). Then the effective contraction factor is $\gamma(1+\lambda_{sym}\mu)$, and:*

(i) If $\lambda_{sym}\mu \geq 1/\gamma - 1$: the operator is not *a contraction;*

(ii) If $\lambda_{sym}\mu < 1/\gamma - 1$: the operator is *a contraction with factor $\gamma(1 + \lambda_{sym}\mu) > \gamma$.*

The boundary $\gamma(1 + \lambda_{sym}\mu) = 1$ is sharp. Machine-verified in Lean 4: `CSBAPR-Counterproof.lean` (`T_bad_sym_not_contraction`, `T_bad_sym_contraction_below_threshold`).

Proof sketch. When $\Gamma_{\text{sym}}(Q) = C - \mu Q$, the penalty term $-\lambda_{\text{sym}}\Gamma_{\text{sym}}(Q)$ contributes $+\lambda_{\text{sym}}\mu Q$ to the operator. The simplified operator becomes $\mathcal{T}_{\text{bad}}(Q) = \gamma(1 + \lambda_{\text{sym}}\mu) \cdot Q + R_0$ for constant R_0. Taking witnesses $Q_1 = 1, Q_2 = 0$: $|\mathcal{T}_{\text{bad}}(1) - \mathcal{T}_{\text{bad}}(0)| = \gamma(1 + \lambda_{\text{sym}}\mu) \geq 1$. □

Remark 4.26 (Structural unity of frozen-parameter counterexamples)**.** *All three counterexamples in this paper series share the same algebraic structure:*

Paper	***Non-frozen term***	***Factor***	***Lean file***
RE-SAC	$\kappa(Q) = \lambda Q$	$\gamma + \lambda$	`RESAC-Counterproof.lean`
BAPR	$\rho(Q)$ + *reward gap* Δ	$\gamma + \lambda\Delta$	`BAPR-Counterproof.lean`
CS-BAPR	$\Gamma_{sym}(Q)$ *feedback*	$\gamma(1 + \lambda_{sym}\mu)$	`CSBAPR-Counterproof.lean`

In every case, the frozen penalty cancels exactly in $\mathcal{T}(Q_1) - \mathcal{T}(Q_2)$ (yielding factor γ), but a Q-dependent version introduces a coupling term that pushes the factor above γ—and past 1 when the coupling is strong enough. The frozen-parameter design is therefore necessary *for contraction in all three frameworks.*

Remark 4.27 (Tractable approximation: shared Γ_{sym} across modes)**.** *The Lean 4 proof indexes Γ_{sym} by mode h (i.e., $\Gamma_{sym}^h(s, a)$), allowing mode-specific symbolic penalties. In the implementation, we use a* single shared *Γ_{sym} across all modes, since the SINDy formula f_{sym} is designed to be causally invariant (via IRM filtering) and therefore mode-independent. This is a conservative approximation: the Lean proof holds for* arbitrary *values of Γ_{sym}^h; using identical values across modes is a strict special case. The approximation affects the quality of the fixed point (how well Γ_{sym} captures mode-specific Jacobian deviations) but not the existence or uniqueness of the fixed point.*

4.6 Component composition: from pillars to bound hypotheses

Before stating the main OOD theorem, we show how the three pillars combine to satisfy its hypothesis—specifically, the linear derivative error growth condition.

Proposition 4.28 (Component hypotheses imply derivative bound)**.** *Under Assumptions 4.2–4.4, if:*

(i) f_{real} has approximately constant derivative (smooth physical law with bounded second derivative $\|D^2 f_{real}\| \leq M$),

(ii) The policy has ε-Jacobian Consistency at the training boundary x_0 (from SINDy + training, Theorem 4.16),

(iii) The policy has L_{eff}-Arithmetic Inductive Bias (from NAU/NMU, Remark 4.14),

then: $\forall x,\ \|D\pi(x) - Df_{real}(x)\| \le (\varepsilon + M \cdot \|x - x_0\|) + L_{eff} \cdot \|x - x_0\|$.

In the idealized case where f_{real} *has exactly constant derivative* ($M = 0$, *e.g., linear physics), this simplifies to* $\|D\pi(x) - Df_{real}(x)\| \le \varepsilon + L_{eff} \cdot |x - x_0|$, *which is precisely the hypothesis of the quadratic OOD bound. (Lean:* `deriv_error_bound_from_components`*, line 157.)*

Proposition 4.29 (n-D composition: from pillars to path-level hypothesis)**.** *Under Assumptions 4.2–4.5, if:*

(i) π *has* L_{eff}*-Arithmetic Inductive Bias in the n-D sense (Definition 4.7),*

(ii) π *is* ε*-Jacobian Consistent with* f_{real} *on the segment from* x_0 *to* x_{ood} *(Assumption 4.5),*

(iii) f_{real} *has* M*-bounded second Fréchet derivative:* $\|D^2 f_{real}(x)\| \le M$,

then the path-level error derivative satisfies: $\|e'_{path}(t)\| \le (\varepsilon + M\|d\|)\|d\| + L_{eff}\|d\|^2 \cdot t$ *for* $t \in [0, 1)$, *where* $d = x_{ood} - x_0$ *and* $e_{path}(t) = \pi(x_0 + td) - f_{real}(x_0 + td)$. *In the idealized case* $M = 0$, *this reduces to* $\varepsilon\|d\| + L_{eff}\|d\|^2 \cdot t$, *which is precisely the hypothesis of Theorem 4.35 with* $\varepsilon_d = \varepsilon\|d\|$ *and* $L_d = L_{eff}\|d\|^2$.

This n-D composition proposition provides the formal bridge between the three pillars (which supply ε, L_{eff}, and the $M = 0$ idealization) and the path-level hypothesis consumed by the fencing theorems. The causal chain is: **NAU/NMU** (Pillar I) provides finite L_{eff}; **SINDy** (Pillar II) provides small $\varepsilon = \varepsilon_1 + \varepsilon_2$; **IRM** (Pillar III) tightens ε by replacing $\varepsilon_1^{\text{raw}}$ with $\varepsilon_1^{\text{irm}} \le \varepsilon_1^{\text{raw}}$; and **RL training** provides small δ. All four quantities enter the final bound.

Remark 4.30 (On the nature of the theoretical contribution)**.** *The algebraic core of the OOD bound is an application of the Mean Value Theorem and Grönwall-type fencing. The theoretical contribution lies not in the calculus but in the* structural design insight*: identifying that the specific combination of NAU/NMU architecture, SINDy alignment, and IRM filtering makes the MVT hypotheses* satisfiable*—and proving that the standard ReLU architecture makes them* provably unsatisfiable*. The Lean 4 verification serves as a design audit tool, forcing every structural assumption to be explicit, much as it did for the frozen-parameter requirement in BAPR [1].*

4.7 Main theorem: OOD extrapolation stability

We now state the main theoretical result, beginning with the key building blocks.

Lemma 4.31 (Linear error growth via MVT)**.** *Let* $\pi, f_{real} : \mathbb{R} \to \mathbb{R}$ *be differentiable, and suppose* $\|\pi'(x) - f'_{real}(x)\| \le C$ *for all* $x \in [a, b)$. *Then* $\|(\pi(b) - f_{real}(b)) - (\pi(a) - f_{real}(a))\| \le C \cdot (b - a)$. *(Lean:* `error_growth_linear`*, line 65. Proof applies Mathlib's* `norm_image_sub_le_of_norm_deriv_right_le_segment` *to the error function* $e(x) = \pi(x) - f_{real}(x)$*.)*

This fundamental lemma establishes that bounded derivative error implies bounded error growth. Combined with the base accuracy δ, it gives the linear OOD bound. The quadratic generalization below replaces the constant C with a *growing* derivative error $\varepsilon + L(x - a)$.

Theorem 4.32 (1D OOD Extrapolation Stability Bound)**.** *Let* $\pi, f_{real} : \mathbb{R} \to \mathbb{R}$ *be differentiable. If:*

(i) $\|\pi(a) - f_{real}(a)\| \le \delta$ *(base accuracy),*

(ii) $\forall x \in [a, b),\ \|\pi'(x) - f'_{real}(x)\| \le \varepsilon + L(x - a)$ *(growing derivative error),*

then: $\|\pi(b) - f_{real}(b)\| \le \delta + \varepsilon(b - a) + \frac{L}{2}(b - a)^2$.
(Lean: `scbapr_ood_stability_bound`*, line 112. Proof uses Mathlib's fencing theorem.)*

Proof sketch. Define the error function $e(x) = \pi(x) - f_{\text{real}}(x)$. Since $\|e(a)\| \le \delta$ and $\|e'(x)\| \le \varepsilon + L(x - a)$ for $x \in [a, b)$, apply Mathlib's Grönwall-style fencing lemma (`image_norm_le_of_norm_deriv_right_le_deriv_l` with envelope $g(x) = \delta + \varepsilon(x - a) + \frac{L}{2}(x - a)^2$. Verify: $g(a) = \delta \ge \|e(a)\|$ and $g'(x) = \varepsilon + L(x - a) \ge \|e'(x)\|$. The conclusion $\|e(b)\| \le g(b)$ gives the bound. □

Theorem 4.33 (n-D Linear OOD Bound)**.** *For $\pi, f_{real} : E \to F$ in normed spaces, if $\|\pi(x_0) - f_{real}(x_0)\| \le \delta$ and $\forall z,\ \|D\pi(z) - Df_{real}(z)\| \le C$, then:*

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \le \delta + C\|x_{ood} - x_0\|. \tag{14}$$

(Lean: `scbapr_ood_linear_nD`, line 413. Proof uses Mathlib's convex Mean Value Theorem.)

Proof sketch. Define $e(x) = \pi(x) - f_{\text{real}}(x)$, so $De(x) = D\pi(x) - Df_{\text{real}}(x)$ with $\|De(x)\| \le C$. Apply Mathlib's convex MVT (`Convex.norm_image_sub_le_of_norm_fderiv_le`): $\|e(x_{\text{ood}}) - e(x_0)\| \le C\|x_{\text{ood}} - x_0\|$. Triangle inequality: $\|e(x_{\text{ood}})\| \le \|e(x_0)\| + C\|x_{\text{ood}} - x_0\| \le \delta + C\|d\|$. □

Lemma 4.34 (Path-level fencing theorem)**.** *Let $e_{path} : \mathbb{R} \to F$ be the error function along a line segment $[0, 1]$ (typically $e_{path}(t) = \pi(x_0 + t \cdot d) - f_{real}(x_0 + t \cdot d)$). If $\|e_{path}(0)\| \le \delta$ and $\|e'_{path}(t)\| \le \varepsilon_d + L_d \cdot t$ for $t \in [0, 1)$, then: $\|e_{path}(1)\| \le \delta + \varepsilon_d + L_d/2$. This is the F-valued generalization of the 1D bound (Theorem 4.32), applied to the composed path. (Lean: `scbapr_ood_quadratic_path`, line 629.)*

The proof chain is: **1D fencing** (Theorem 4.32) → **path-level fencing** (Lemma 4.34) → **n-D bound** (Theorem 4.35 below), where the final step substitutes $\varepsilon_d = \varepsilon\|d\|$ and $L_d = L\|d\|^2$.

Theorem 4.35 (n-D Quadratic OOD Bound (Main Result))**.** *For $\pi, f_{real} : E \to F$, if along the line segment from x_0 to x_{ood}:*

(i) $\|\pi(x_0) - f_{real}(x_0)\| \le \delta$,

(ii) The derivative error grows linearly: $\|e'(t)\| \le \varepsilon\|d\| + L\|d\|^2 \cdot t$ *for* $t \in [0, 1)$,

where $d = x_{ood} - x_0$, then:

$$\|\pi(x_{ood}) - f_{real}(x_{ood})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2. \tag{15}$$

(Lean: `scbapr_ood_quadratic_nD`, line 672.)

Proof sketch. Parametrize the line segment as $\gamma(t) = x_0 + t \cdot d$ for $t \in [0, 1]$ and define $e_{\text{path}}(t) = \pi(\gamma(t)) - f_{\text{real}}(\gamma(t))$. Then $\|e_{\text{path}}(0)\| \le \delta$ and $\|e'_{\text{path}}(t)\| \le \varepsilon\|d\| + L\|d\|^2 t$ by hypothesis (ii). Apply the path-level fencing lemma (Lemma 4.34) with $\varepsilon_d = \varepsilon\|d\|$ and $L_d = L\|d\|^2$: $\|e_{\text{path}}(1)\| \le \delta + \varepsilon_d + L_d/2 = \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Since $\gamma(1) = x_{\text{ood}}$, this gives $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \le \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Full proof in Appendix B. □

Remark 4.36 (Engineering interpretation)**.** *The bound (15) decomposes OOD error into three controllable terms, each addressed by a specific CS-BAPR component:*

Term	***Meaning***	***Controlled by***
δ	*Training-domain accuracy*	*Standard RL training*
$\varepsilon\|d\|$	*SINDy quality × distance*	*Pillar II (SINDy) + III (IRM)*
$\frac{L}{2}\|d\|^2$	*Architectural extrapolation*	*Pillar I (NAU: L=0, NMU: L=2|c|)*

With NAU ($L = 0$), the bound is linear *in distance. With NMU, it is* quadratic*. With ReLU, no finite L exists within this framework (Theorem 4.12), so the bound cannot be instantiated. This hierarchy provides a principled criterion for architecture selection: the trade-off is expressiveness (NMU can represent quadratic functions that NAU cannot) vs. extrapolation tightness ($L = 0$ for NAU vs. $L = 2|c|$ for NMU).*

Remark 4.37 (Necessity of each pillar)**.** *Each pillar is necessary in a precise sense, as removing any one causes a specific term in the OOD bound to become uncontrolled:*

- ***Without NAU/NMU (Pillar I):*** *$L = \infty$ for ReLU (Theorem 4.12), and the quadratic term $\frac{L}{2}\|d\|^2 \to \infty$. The bound is* vacuous.

- ***Without SINDy (Pillar II):*** *The Jacobian Consistency error ε is uncontrolled (no gradient alignment signal), and the linear term $\varepsilon\|d\|$ dominates. The bound* exists *but is too loose to be useful.*

- ***Without IRM (Pillar III):*** *The bound still holds with $\varepsilon_{raw} \geq \varepsilon_{irm}$, but may capture spurious correlations. IRM is the only non-necessary pillar in a strict mathematical sense—it tightens ε but is not required for the bound to be finite.*

Thus, Pillars I and II are structurally necessary *for finite OOD bounds, while Pillar III provides* quantitative tightening. *The graceful degradation hierarchy (Remark 4.39) ensures that partial configurations remain useful.*

4.8 CS-BAPR algorithm

The complete CS-BAPR algorithm operates in two phases:

Phase 0: SINDy Pre-Identification. Before RL training begins, the agent collects exploration trajectories using a random policy, fits a SINDy model to discover sparse dynamics, and optionally applies IRM filtering across multiple environments. The resulting symbolic formula f_{sym} is frozen as a PyTorch module with detached gradients.

Phase 1: CS-BAPR Training. The training loop extends BAPR with two additions: (1) the symbolic consistency penalty Γ_{sym} in the Q-loss (Eq. (11)), and (2) the Jacobian Consistency Loss $\mathcal{L}_{\text{AC}}$ (Eq. (7)) in the policy loss.

Algorithm 1 CS-BAPR: Causal-Symbolic BAPR

Phase 0: SINDy Pre-Identification
Collect exploration trajectories $\{(s_t, a_t, s_{t+1})\}$ using random policy.
Fit SINDy: $\xi = \arg\min \|\dot{X} - \Theta(X)\xi\|_2 + \lambda\|\xi\|_1$.
(Optional) IRM filter: select ξ with minimum cross-environment variance.
Create frozen PyTorch wrapper: $f_{\text{sym}} \leftarrow$ `SINDyTorchWrapper`(ξ).

Phase 1: CS-BAPR Training Loop
Initialize: NAU/NMU Actor π_θ, Ensemble critics $\{Q_{\phi_k}\}$, BOCD belief ρ.
for each training iteration **do**
  Collect transitions; store in $\mathcal{D}$.
  Compute surprise ξ_t; update BOCD belief ρ (frozen during backup).
  Compute λ_w and β_{eff} from belief (inherited from BAPR).
  for $n = 1, \ldots, N_{\text{updates}}$ **do**
    Sample mini-batch from $\mathcal{D}$.
    // Critic update (with Γ_{sym}):
    $\Gamma_{\text{sym}} \leftarrow \|\nabla_s \pi_\theta(s') - \nabla_s f_{\text{sym}}(s')\|^2$ (SINDy gradients detached).
    $y \leftarrow r + \gamma(\bar{Q}_{\text{target}} - \alpha \log \pi - \lambda_{\text{epi}}\Gamma_{\text{epi}} - \lambda_{\text{sym}}\Gamma_{\text{sym}} - \kappa)$.
    Update ϕ_k by minimizing $\mathcal{L}_Q + \beta_{\text{ood}}\sigma_{\text{ens}}$.
    // Actor update (with $\mathcal{L}_{AC}$):
    $\mathcal{L}_\pi \leftarrow -(\bar{Q} + \beta_{\text{eff}}\sigma_{\text{ens}} - \alpha \log \pi) + \mu_{\text{jac}} \cdot \mathcal{L}_{\text{AC}}$.
    Update θ by minimizing $\mathcal{L}_\pi$ (with gradient clipping).
    Soft update target networks.
  end for
end for

Remark 4.38 (Frozen-parameter requirements)**.** *CS-BAPR requires four frozen-parameter conditions for contraction:*

(i) κ_{ale} frozen (from RE-SAC target weights);

(ii) Γ_{epi} frozen (from target Q-ensemble);

(iii) ρ frozen during backup (from BOCD, updated between iterations);

(iv) Γ_{sym} frozen (SINDy coefficients fixed throughout training). ***[CS-BAPR new]***

Violating any one of these conditions breaks the contraction guarantee, as established by the machine-verified counterexamples in BAPR [1], RE-SAC [2], and CS-BAPR (Theorem 4.25).

Remark 4.39 (Graceful degradation)**.** *CS-BAPR is designed to degrade gracefully when its assumptions are not met:*

- ***SINDy fails*** *($\varepsilon_1 > \varepsilon_{threshold}$): Γ_{sym} and $\mathcal{L}_{AC}$ are disabled; the algorithm reduces to BAPR with a standard MLP or NAU/NMU actor, retaining all BAPR convergence guarantees.*
- ***No multiple environments****: IRM filtering is skipped; single-environment SINDy formula used directly, with ε_{raw} replacing ε_{irm} in the bound.*
- ***NAU/NMU disabled****: The architecture reverts to ReLU; the OOD bound no longer holds, but the Bellman contraction guarantee is unaffected (it depends only on frozen penalties, not on the actor architecture).*

This hierarchical fallback ensures that CS-BAPR is never worse *than BAPR, even when its additional components cannot be applied.*

Remark 4.40 (Computational overhead)**.** *CS-BAPR adds three sources of overhead relative to BAPR:*

- ***Phase 0 (SINDy, one-time):*** *50 exploration episodes + PySINDy fitting, typically <5 minutes. Amortized over the full training, this is negligible.*
- ***$\mathcal{L}_{AC}$ (per gradient step):*** *One additional* `autograd` *call to compute $\nabla_s \pi_\theta(s)$. For the action-dimension output ($\dim(\mathcal{A}) \ll \dim(\mathcal{S})$), this is $O(\dim(\mathcal{S}))$ per sample, comparable to a single backward pass.*
- ***NAU regularization (per step):*** *$O(1)$—a scalar penalty on weight magnitudes.*

In wall-clock time, CS-BAPR is approximately 15–20% slower than BAPR (the dominant cost remains the ensemble critic's K forward/backward passes). The Jacobian computation can be further accelerated via JVP with random vectors for high-dimensional state spaces, reducing cost from $O(\dim(\mathcal{S})^2)$ to $O(\dim(\mathcal{S}))$.

5 Experiments

We evaluate CS-BAPR on continuous control tasks designed to test OOD extrapolation capability, addressing four questions: **(Q1)** Does CS-BAPR extrapolate where standard RL fails? **(Q2)** Is each pillar (NAU, SINDy, IRM) necessary? **(Q3)** Does empirical OOD error match the theoretical bound? **(Q4)** How does the NAU vs. ReLU architectural choice affect extrapolation?

5.1 Experimental setup

Environments. We evaluate on environments with known physical dynamics amenable to SINDy identification: Pendulum-v1 (analytic dynamics $\ddot{\theta} = -g/l \sin\theta + u/ml^2$), and MuJoCo locomotion tasks (Hopper, HalfCheetah) with controlled OOD evaluation at extrapolated state magnitudes.

OOD protocol. We train on a bounded state region D_{train} and evaluate at progressively increasing distances $\|d\| = \{1\times, 2\times, 4\times, 8\times\}$ the training range, measuring the policy's prediction error against ground-truth dynamics.

Baselines.

(i) **SAC** [3]: standard ReLU policy;

(ii) **RE-SAC** [2]: robust ensemble (ReLU);

(iii) **BAPR** [1]: full BAPR framework (ReLU);

(iv) **CS-BAPR (ours)**: NAU/NMU + SINDy + IRM.

5.2 Main results (Q1)

(OOD extrapolation comparison results to be inserted after experiments are completed. We expect CS-BAPR to maintain bounded error at 4×–8× training range while ReLU-based baselines exhibit unbounded error growth.)

5.3 Ablation study (Q2)

To isolate each pillar's contribution:

- **CS-BAPR w/o NAU**: Replace NAU/NMU actor with standard MLP (ReLU). Tests architectural contribution.
- **CS-BAPR w/o SINDy**: Remove Γ_{sym} and $\mathcal{L}_{\text{AC}}$. Tests symbolic guidance.
- **CS-BAPR w/o IRM**: Use single-environment SINDy formula. Tests causal filtering.

(Ablation results to be inserted.)

5.4 OOD error vs. theoretical bound (Q3)

We plot empirical OOD error $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\|$ against the theoretical bound $\delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$ as a function of distance $\|d\|$, using measured values of δ, ε, and L.

(Error vs. bound plots to be inserted.)

5.5 NAU vs. ReLU architecture (Q4)

We compare the derivative Lipschitz constant and OOD error between NAU/NMU and ReLU actors trained on identical data, validating the architectural hierarchy established in Remark 4.13.

(Architecture comparison to be inserted.)

6 Conclusion

This paper presents CS-BAPR, a framework that addresses the fundamental challenge of OOD extrapolation in reinforcement learning through three synergistic pillars: Neural Arithmetic Units for architecturally guaranteed derivative regularity, SINDy symbolic discovery for physics-informed gradient alignment, and IRM causal filtering for invariant law selection. Our main theoretical contribution is the OOD Extrapolation Stability Bound—the first machine-verified polynomial bound on RL policy error in out-of-distribution regions—along with the ReLU Impossibility Theorem establishing that standard neural architectures are structurally incapable of satisfying such bounds.

All theoretical results are machine-verified in Lean 4 with no `sorry` (954 lines across `CSBAPR.lean`, encompassing 34 definitions, theorems, and lemmas). The verification covers the full pipeline from SINDy identification through IRM filtering to the final n-dimensional quadratic OOD bound, and extends the BAPR Bellman contraction guarantee to the symbolic-penalty-augmented operator.

Limitations. CS-BAPR's applicability is bounded by its assumptions. First, the OOD bound requires that the target environment admits SINDy-identifiable dynamics (Assumption 4.2); pixel-based environments such as Atari, or environments with highly discontinuous dynamics (e.g., contact-rich manipulation), fall outside this scope. Second, the quadratic bound is tight only when the effective Lipschitz constant L_{eff} is small; for complex multi-layer architectures, the composed L_{eff} may be large enough that the quadratic term dominates at moderate distances, reducing the practical

benefit. Third, the frozen-SINDy design (required for contraction) means that if the environment dynamics genuinely change (as in BAPR's piecewise-stationary setting), the SINDy formula becomes stale and must be re-identified—a procedure that currently requires pausing training for a new Phase 0. Developing an online SINDy refinement protocol that preserves the frozen-parameter guarantee within each backup (e.g., updating coefficients only between training epochs) is an important direction for future work.

In future work, we plan to extend CS-BAPR to high-dimensional visual observation spaces through latent-space SINDy, investigate online SINDy refinement with formal re-verification of frozen-parameter conditions, explore the integration of CS-BAPR's extrapolation capabilities with distributional RL for risk-sensitive OOD deployment, and study tighter Lipschitz analysis for composed NAU/NMU architectures (e.g., via LipSDP or spectral normalization bounds).

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 72371251), the Natural Science Foundation for Distinguished Young Scholars of Hunan Province (Grant No. 2024JJ2080), and the Key Research and Development Program of Hunan Province of China (Grant No. 2024JK2007).

References

[1] Zhang Yifan and Zheng Liang, "BAPR: Bayesian amnesic piecewise-robust reinforcement learning for non-stationary continuous control," *arXiv preprint*, 2025.

[2] Zhang Yifan and Zheng Liang, "RE-SAC: Disentangling aleatoric and epistemic risks in bus fleet control: A stable and robust ensemble DRL approach," *arXiv preprint*, 2025.

[3] Haarnoja Tuomas, Zhou Aurick, Abbeel Pieter, and Levine Sergey, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.

[4] Adams Ryan P. and MacKay David J.C., "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.

[5] Lee Joonho, Hwangbo Jemin, Wellhausen Lorenz, Koltun Vladlen, and Hutter Marco, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.

[6] Wiesemann Wolfram, Kuhn Daniel, and Rustem Berç, "Robust markov decision processes," *Mathematics of Operations Research*, vol. 38, no. 1, pp. 153–183, 2013.

[7] Iyengar Garud N., "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.

[8] Zhou Ruida, Liu Tao, Cheng Min, Kalathil Dileep, Kumar P. R., and Tian Chao, "Natural actor-critic for robust reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 30 424–30 438.

[9] Panaganti Kishan, Wierman Adam, and Mazumdar Eric, "Model-free robust ϕ-divergence reinforcement learning using both offline and online data," in *International Conference on Machine Learning (ICML)*, 2024, pp. 39 389–39 459.

[10] Ghosh Dibya, Rahme Jad, Kumar Aviral, Zhang Amy, Adams Ryan P., and Levine Sergey, "Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 25 502–25 515.

[11] Trask Andrew, Hill Felix, Reed Scott E., Rae Jack, Dyer Chris, and Blunsom Phil, "Neural arithmetic logic units," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 8035–8044.

[12] Madsen Andreas and Johansen Alexander R., "Neural arithmetic units," in *International Conference on Learning Representations (ICLR)*, 2020.

[13] Sahoo Subham, Lampert Christoph, and Martius Georg, "Learning equations for extrapolation and control," in *International Conference on Machine Learning (ICML)*, 2018, pp. 4442–4450.

[14] Brunton Steven L., Proctor Joshua L., and Kutz J. Nathan, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.

[15] Vajdi Amin, Zolman Nicholas, Gu Bolun, Paull Liam, and Bhatt Meghdeep, "SINDy-RL: Interpretable and efficient model-based reinforcement learning," *arXiv preprint arXiv:2403.09110*, 2024.

[16] Udrescu Silviu-Marian and Tegmark Max, "AI Feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, p. eaay2631, 2020.

[17] Arjovsky Martin, Bottou Léon, Gulrajani Ishaan, and Lopez-Paz David, "Invariant risk minimization," *arXiv preprint arXiv:1907.02893*, 2019.

[18] Schölkopf Bernhard, Locatello Francesco, Bauer Stefan, Ke Nan Rosemary, Kalchbrenner Nal, Goyal Anirudh, and Bengio Yoshua, "Toward causal representation learning," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 612–634, 2021.

[19] Zeng Yan, Cai Ruichu, Sun Fuchun, Huang Libo, and Hao Zhifeng, "A survey on causal reinforcement learning," *arXiv preprint arXiv:2302.05209*, 2023.

[20] Rowan Taniya, "On the failure of ReLU activation for physics-informed machine learning," *arXiv preprint*, 2023.

[21] Zhang Yifan and Zheng Liang, "Single agent robust deep reinforcement learning for bus fleet control," *arXiv preprint*, 2025.

[22] An Gaon, Moon Seungyong, Kim Jang-Hyun, and Song Hyun Oh, "Uncertainty-based offline reinforcement learning with diversified q-ensemble," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 7436–7447.

[23] Ji Tianying, Luo Yu, Sun Fuchun, Zhan Xianyuan, Zhang Jianwei, and Xu Huazhe, "Seizing serendipity: Exploiting the value of past success in off-policy actor-critic," in *International Conference on Machine Learning (ICML)*, 2024, pp. 21 672–21 718.

[24] Kirsch Louis, Harrison James, Sohl-Dickstein Jascha, and Metz Luke, "General-purpose in-context learning by meta-learning transformers," *arXiv preprint arXiv:2212.04458*, 2022.

[25] Osogami Takayuki, "Robustness and risk-sensitivity in markov decision processes," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, 2012, pp. 233–241.

[26] Shalev-Shwartz Shai and Ben-David Shai, *Understanding Machine Learning: From Theory to Algorithms.* Cambridge University Press, 2014.

A Lean 4 Proof Overview

The formal verification is contained in a single file `CSBAPR.lean` (954 lines, 0 `sorry`) within the `proof/CS-BAPR/` directory of the project repository. The file is organized into nine parts:

Part	Content	Lines	Key Theorem
I	Core definitions	33–57	`IsJacobianConsistent`, `HasArithmeticInductiveBias`
II	1D OOD extrapolation	59–188	`scbapr_ood_stability_bound`
III	CS-BAPR Bellman contraction	189–395	`csbapr_contraction`
IV	Belief update preservation	346–386	`csbapr_contraction_after_update`
V	n-D generalization	394–443	`scbapr_ood_linear_nD`
VI	SINDy formalization	456–595	`sindy_to_ood_bound_nD`
VII	n-D quadratic OOD bound	597–697	`scbapr_ood_quadratic_nD`
VIII	IRM causal filtering	699–774	`irm_to_ood_bound_nD`
IX	NAU/NMU structural guarantees	776–953	`relu_derivative_not_lipschitz`

The proof relies on Mathlib's calculus library, particularly:

- `norm_image_sub_le_of_norm_deriv_right_le_segment` — 1D MVT-based segment bound.
- `image_norm_le_of_norm_deriv_right_le_deriv_boundary'` — Grönwall-style fencing theorem (1D quadratic bound).
- `Convex.norm_image_sub_le_of_norm_fderiv_le` — Convex set MVT for n-D linear bound.
- `image_norm_le_of_norm_deriv_right_le_deriv_boundary` — F-valued fencing theorem (n-D quadratic bound).

B Complete Proofs

This section provides full mathematical proofs for the key theorems stated in the main text. All proofs are also machine-verified in Lean 4.

B.1 Proof of Theorem 4.32 (1D OOD Stability Bound)

Proof. Define $e : \mathbb{R} \to \mathbb{R}$ by $e(x) = \pi(x) - f_{\text{real}}(x)$. Since π and f_{real} are differentiable, so is e, with $e'(x) = \pi'(x) - f'_{\text{real}}(x)$.

By hypothesis, $\|e(a)\| \le \delta$ and $\|e'(x)\| \le \varepsilon + L(x-a)$ for all $x \in [a,b)$.

Define the comparison function $g : [a,b] \to \mathbb{R}$ by $g(x) = \delta + \varepsilon(x-a) + \frac{L}{2}(x-a)^2$.

We verify: (i) $g(a) = \delta \ge \|e(a)\|$; (ii) $g'(x) = \varepsilon + L(x-a) \ge \|e'(x)\|$ for $x \in [a,b)$.

By the Grönwall-style fencing theorem (for F-valued functions with real envelope, formalized in Mathlib as `image_norm_le_of_norm_deriv_right_le_deriv_boundary'`), we conclude $\|e(x)\| \le g(x)$ for all $x \in [a,b]$.

Evaluating at $x = b$: $\|e(b)\| \le g(b) = \delta + \varepsilon(b-a) + \frac{L}{2}(b-a)^2$. □

B.2 Proof of Theorem 4.33 (n-D Linear Bound)

Proof. Define $e : E \to F$ by $e(x) = \pi(x) - f_{\text{real}}(x)$. Then $De(x) = D\pi(x) - Df_{\text{real}}(x)$, and $\|De(x)\|_{\text{op}} \le C$ for all x by hypothesis.

The convex hull of $\{x_0, x_{\text{ood}}\}$ is the line segment $[x_0, x_{\text{ood}}]$, which is convex. By the n-dimensional Mean Value Theorem for normed spaces (Mathlib: `Convex.norm_image_sub_le_of_norm_fderiv_le`):

$$\|e(x_{\text{ood}}) - e(x_0)\| \le C \cdot \|x_{\text{ood}} - x_0\|.$$

Triangle inequality:

$$\|e(x_{\text{ood}})\| \leq \|e(x_0)\| + \|e(x_{\text{ood}}) - e(x_0)\| \leq \delta + C\|d\|.$$

□

B.3 Proof of Theorem 4.35 (n-D Quadratic OOD Bound — Main Result)

Proof. Let $d = x_{\text{ood}} - x_0$ and parametrize the line segment as $\gamma(t) = x_0 + td$ for $t \in [0, 1]$.

Define the pullback error function $e_{\text{path}} : [0, 1] \to F$ by $e_{\text{path}}(t) = \pi(\gamma(t)) - f_{\text{real}}(\gamma(t))$.

Step 1 (Boundary condition): $\|e_{\text{path}}(0)\| = \|\pi(x_0) - f_{\text{real}}(x_0)\| \leq \delta$.

Step 2 (Derivative bound): By the chain rule, $e'_{\text{path}}(t) = (D\pi(\gamma(t)) - Df_{\text{real}}(\gamma(t))) \cdot d$, so $\|e'_{\text{path}}(t)\| \leq \|D\pi(\gamma(t)) - Df_{\text{real}}(\gamma(t))\| \cdot \|d\|$. By hypothesis (ii), this is bounded by $\varepsilon\|d\| + L\|d\|^2 \cdot t$.

Step 3 (Fencing): Define the comparison function $g(t) = \delta + \varepsilon\|d\| \cdot t + \frac{L\|d\|^2}{2} \cdot t^2$ with $g(0) = \delta \geq \|e_{\text{path}}(0)\|$ and $g'(t) = \varepsilon\|d\| + L\|d\|^2 t \geq \|e'_{\text{path}}(t)\|$. By the fencing theorem (Mathlib: `image_norm_le_of_norm_deriv_right_le_deriv_boundary`), $\|e_{\text{path}}(t)\| \leq g(t)$ for all $t \in [0, 1]$.

Step 4 (Evaluation): $\|e_{\text{path}}(1)\| \leq g(1) = \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. Since $\gamma(1) = x_{\text{ood}}$, this gives $\|\pi(x_{\text{ood}}) - f_{\text{real}}(x_{\text{ood}})\| \leq \delta + \varepsilon\|d\| + \frac{L}{2}\|d\|^2$. □

B.4 Proof of Theorem 4.23 (CS-BAPR Contraction)

Proof. Fix Q_1, Q_2 with $\|Q_1 - Q_2\|_\infty \leq \varepsilon$. For each mode h and state-action (s, a):

$$\begin{aligned} &|\mathcal{T}_h^{CS} Q_1(s, a) - \mathcal{T}_h^{CS} Q_2(s, a)| \\ &= \gamma \left| \sum_{s'} P_h(s'|s, a)(V^{Q_1}(s') - V^{Q_2}(s')) \right| \end{aligned}$$

where the frozen penalty terms (Γ^h_{epi}, Γ^h_{sym}, κ_{ale}) cancel exactly because they do not depend on Q_1 or Q_2.

Since $|V^{Q_1}(s') - V^{Q_2}(s')| \leq \|Q_1 - Q_2\|_\infty = \varepsilon$ and $\sum_{s'} P_h(s'|s, a) = 1$:

$$|\mathcal{T}_h^{CS} Q_1(s, a) - \mathcal{T}_h^{CS} Q_2(s, a)| \leq \gamma\varepsilon.$$

For the belief-weighted mixture: $|\mathcal{T}_\rho^{CS} Q_1(s, a) - \mathcal{T}_\rho^{CS} Q_2(s, a)| \leq \sum_h \rho(h) \cdot \gamma\varepsilon = \gamma\varepsilon$ since $\sum_h \rho(h) = 1$. Taking the supremum over (s, a) gives $\|\mathcal{T}_\rho^{CS} Q_1 - \mathcal{T}_\rho^{CS} Q_2\|_\infty \leq \gamma\varepsilon = \gamma\|Q_1 - Q_2\|_\infty$. Banach's fixed-point theorem gives the unique fixed point. □

C Remaining Proofs

This section provides proofs for theorems stated in the main text that were not covered in Appendix B. Each proof is paired with its Lean 4 cross-reference.

C.1 Proof of Lemma 4.31 (Linear Error Growth)

Proof. Define $e(x) = \pi(x) - f_{\text{real}}(x)$. Then $e'(x) = \pi'(x) - f'_{\text{real}}(x)$ and $\|e'(x)\| \leq C$ for $x \in [a, b)$. Apply Mathlib's segment MVT (`norm_image_sub_le_of_norm_deriv_right_le_segment`) to e on $[a, b]$:

$$\|e(b) - e(a)\| \leq C \cdot (b - a).$$

Expanding: $\|(\pi(b) - f_{\text{real}}(b)) - (\pi(a) - f_{\text{real}}(a))\| \leq C(b - a)$.

Lean: `error_growth_linear`, line 65. □

C.2 Proof of Lemma 4.34 (Path-Level Fencing)

Proof. $e_{\text{path}} : [0,1] \to F$ satisfies $\|e_{\text{path}}(0)\| \le \delta$ and $\|e'_{\text{path}}(t)\| \le \varepsilon_d + L_d \cdot t$ for $t \in [0,1)$.

Define the comparison function $g(t) = \delta + \varepsilon_d \cdot t + \frac{L_d}{2} t^2$. Then $g(0) = \delta \ge \|e_{\text{path}}(0)\|$ and $g'(t) = \varepsilon_d + L_d \cdot t \ge \|e'_{\text{path}}(t)\|$.

By Mathlib's fencing theorem (`image_norm_le_of_norm_deriv_right_le_deriv_boundary`), $\|e_{\text{path}}(t)\| \le g(t)$ for all $t \in [0,1]$.

Evaluating at $t = 1$: $\|e_{\text{path}}(1)\| \le \delta + \varepsilon_d + L_d/2$.

Lean: `scbapr_ood_quadratic_path`, line 629. □

C.3 Proof of Theorem 4.9 (NAU Derivative Constancy)

Proof. NAU computes $y = \sum_j w_j x_j$, so $\partial y / \partial x_i = w_i$ for all input vectors x. Since w_i is a fixed integer weight (independent of x), the derivative function $x \mapsto w_i$ is constant. Formally, $\text{deriv}(\lambda x_i.\ \sum_j w_j x_j, i) = \lambda_.\ w_i$.

Lean: `nau_derivative_is_constant`, line 817. □

C.4 Proof of Corollary 4.17 (Tight SINDy Consistency)

Proof. When the same coefficient set ξ is used both for SINDy identification and for training, the triangle inequality gives: $\|D\pi(x) - Df_{\text{real}}(x)\| \le \|D\pi(x) - Df_{\text{sym}}(x)\| + \|Df_{\text{sym}}(x) - Df_{\text{real}}(x)\| \le \varepsilon_2 + \varepsilon_1$. This is the same bound as Theorem 4.16 but with the explicit requirement that both error measurements use the same $f_{\text{sym}} = \sum_i \xi_i \varphi_i$.

Lean: `sindy_implies_jacobian_consistency_tight`, line 546. □

C.5 Proof of Theorem 4.21 (Full IRM-to-OOD Pipeline)

Proof. Compose in three steps:

1. By IRM Consistency, $\forall x,\ \|Df^e_{\text{real}}(x) - Df_{\text{sym}}(x)\| \le \varepsilon_{\text{irm}}$ in environment e.
2. By the training bound, $\forall x,\ \|D\pi(x) - Df_{\text{sym}}(x)\| \le \varepsilon_{\text{pol}}$.
3. Triangle inequality: $\|D\pi(x) - Df^e_{\text{real}}(x)\| \le \varepsilon_{\text{irm}} + \varepsilon_{\text{pol}}$.

This gives $(\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}})$-Jacobian Consistency with f^e_{real}. Setting $C = \varepsilon_{\text{irm}} + \varepsilon_{\text{pol}}$ in the n-D linear OOD bound (Theorem 4.33):

$$\|\pi(x_{\text{ood}}) - f^e_{\text{real}}(x_{\text{ood}})\| \le \delta + (\varepsilon_{\text{irm}} + \varepsilon_{\text{pol}})\|x_{\text{ood}} - x_0\|.$$

Lean: `irm_to_ood_bound_nD`, line 755. □

C.6 Proof of Corollary 4.12 (Contraction After Belief Update)

Proof. After Bayesian update, $\rho'(h) = \rho(h) \cdot L(h)/Z$ where $L(h) \ge 0$ and $Z = \sum_h \rho(h) L(h) > 0$.

Since $\rho(h) \ge 0$ and $L(h) \ge 0$, we have $\rho'(h) \ge 0$. Since $\sum_h \rho'(h) = \sum_h \rho(h) L(h)/Z = Z/Z = 1$, ρ' is a valid probability distribution.

The proof of Theorem 4.23 only requires $\rho(h) \ge 0$ and $\sum_h \rho(h) = 1$. Since ρ' satisfies both conditions, $\mathcal{T}^{CS\text{-}BAPR}_{\rho'}$ is a γ-contraction.

Lean: `csbapr_contraction_after_update`, line 370. □

D CS-BAPR Operator: Detailed Derivation

D.1 Relationship to BAPR and RE-SAC operators

The CS-BAPR mode-conditional operator (Eq. (11)) extends the BAPR operator with a single additional frozen penalty term $\lambda_{\text{sym}} \cdot \Gamma^h_{\text{sym}}(s,a)$:

Penalty	Source	Origin
$\kappa_{\text{ale}} = \lambda_{\text{ale}} \sum_l \|W_l\|_1$	Frozen target critic weights	RE-SAC
$\Gamma^h_{\text{epi}}(s,a) = \text{Var}(\{Q_{\phi'_k}\})$	Frozen target Q-ensemble	RE-SAC
$\rho(h)$ belief weights	Frozen BOCD posterior	BAPR
$\Gamma^h_{\text{sym}}(s,a) = \|\nabla_s \pi - \nabla_s f_{\text{sym}}\|^2$	Frozen SINDy coefficients	**CS-BAPR**

D.2 Why Γ_{sym} cancels in contraction

In the contraction argument (Lean: `t_mode_pointwise_bound`, line 251), all terms that do not depend on the Q-function being evaluated cancel:

$$\begin{aligned}
&\mathcal{T}_h^{CS} Q_1(s,a) - \mathcal{T}_h^{CS} Q_2(s,a) \\
&= \gamma \sum_{s'} P_h(s'|s,a) \left[V^{Q_1}(s') - V^{Q_2}(s') \right] \\
&\quad \underbrace{-\lambda_{\text{epi}} \Gamma^h_{\text{epi}} + \lambda_{\text{epi}} \Gamma^h_{\text{epi}}}_{=0} \underbrace{-\lambda_{\text{sym}} \Gamma^h_{\text{sym}} + \lambda_{\text{sym}} \Gamma^h_{\text{sym}}}_{=0} \underbrace{-\kappa + \kappa}_{=0}.
\end{aligned} \tag{16}$$

The cancellation requires that Γ_{sym} is *identical* in both $\mathcal{T}_h^{CS} Q_1$ and $\mathcal{T}_h^{CS} Q_2$, which holds precisely because the SINDy coefficients are frozen (not updated based on Q).

D.3 Frozen-parameter requirement for Γ_{sym}

In the implementation, f_{sym} is a `SINDyTorchWrapper` whose coefficients are stored as non-trainable buffers (`register_buffer`). The SINDy gradients are `detach()`-ed in the Jacobian loss computation:

$$\Gamma_{\text{sym}} = \|\nabla_s \pi_\theta(s) - \underbrace{\nabla_s f_{\text{sym}}(s)}_{\text{detached}} \|^2. \tag{17}$$

This ensures that Γ_{sym} depends on the current policy parameters θ (through $\nabla_s \pi_\theta$) but *not* on the Q-function being updated, satisfying the frozen-parameter condition.

E SINDy Implementation Details

E.1 State derivative estimation

SINDy requires state derivatives $\dot{X} = ds/dt$. In the RL setting, we only observe discrete transitions (s_t, a_t, s_{t+1}). We use first-order finite differences:

$$\dot{s}_t \approx \frac{s_{t+1} - s_t}{\Delta t}, \tag{18}$$

where Δt is the environment timestep. For discrete-time SINDy (our default), we directly model $s_{t+1} = f(s_t)$, avoiding derivative estimation entirely.

E.2 SINDy-to-PyTorch bridge

The SINDy model (fitted via PySINDy in NumPy) is converted to a differentiable PyTorch module by reconstructing the polynomial basis function computation in PyTorch:

$$f_{\text{sym}}(x) = \Theta(x) \cdot \xi^T, \tag{19}$$

where $\Theta(x)$ is the polynomial library matrix and ξ are the frozen SINDy coefficients stored as `register_buffer`. This enables automatic differentiation for the Jacobian Consistency Loss without backpropagating through the SINDy coefficients.

E.3 Quality verification

Before proceeding to Phase 1 training, we verify the SINDy quality gate:

$$\varepsilon_1 = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} \|Df_{\text{real}}(x) - Df_{\text{sym}}(x)\| < \varepsilon_{\text{threshold}}. \tag{20}$$

If SINDy fails this gate, the basis library degree is increased or additional exploration data is collected.

F NAU/NMU Architecture Details

F.1 NAU weight discretization

NAU layers use clamped real-valued weights with a sparsity regularizer that drives weights toward $\{-1, 0, 1\}$:

$$\mathcal{L}_{\text{NAU}} = \sum_i \min(|w_i|, |1 - |w_i||). \tag{21}$$

This is a continuous relaxation following Madsen & Johansen [12], where $w_i \in [-1, 1]$ via clamping and the regularizer encourages convergence to the discrete set.

F.2 Actor architecture

The CS-BAPR actor combines a standard feature extractor (2-layer MLP with LeakyReLU) with NAU/NMU output heads:

$$h = \text{LeakyReLU}(W_2 \cdot \text{LeakyReLU}(W_1 \cdot s)), \tag{22}$$

$$a_{\text{linear}} = \text{NAU}(h), \quad L = 0, \tag{23}$$

$$a_{\text{quad}} = \text{NMU}(\text{Linear}(h)), \quad L = 2|c|, \tag{24}$$

$$a = \tanh(\alpha \cdot a_{\text{linear}} + (1 - \alpha) \cdot a_{\text{quad}}), \tag{25}$$

where $\alpha \in [0, 1]$ is a learnable mixing coefficient.

F.3 Effective Lipschitz derivative constant L_{eff}

The effective Lipschitz derivative constant of the composed actor is bounded by:

$$L_{\text{eff}} \leq L_{\text{feat}} \cdot L_{\text{head}} \cdot L_{\tanh}, \tag{26}$$

where each factor corresponds to a composition stage:

- L_{feat}: The feature extractor's Lipschitz constant. For bounded-weight networks with LeakyReLU activations (slope $\alpha_\ell > 0$), this is finite: LeakyReLU is globally Lipschitz with constant $\max(1, \alpha_\ell) = 1$, so $L_{\text{feat}} \leq \prod_l \|W_l\|_{\text{op}}$. This is controlled via weight clamping inherited from the NAU regularizer.
- $L_{\text{head}} = \alpha \cdot 0 + (1 - \alpha) \cdot 2|c| = 2(1 - \alpha)|c|$: The output head's derivative Lipschitz constant, which is a convex combination of NAU ($L = 0$) and NMU ($L = 2|c|$).
- $L_{\tanh} \leq 1$: Since $|\tanh'(x)| \leq 1$ everywhere, the tanh activation does not amplify the Lipschitz constant.

Crucially, the LeakyReLU feature layers contribute a *finite* L_{feat} (unlike ReLU, whose derivative is discontinuous at 0). The composed L_{eff} is therefore finite, preserving the quadratic OOD bound, albeit with a potentially larger constant than the isolated NAU/NMU proofs suggest. The constant can be further tightened via spectral normalization [12] or LipSDP methods.

G IRM Filtering Protocol

G.1 Multi-environment setup

IRM filtering requires at least $K \geq 3$ training environments with distinct dynamics parameters but shared causal structure. For each environment e_k:

1. Collect exploration trajectories $\{(s_t^k, a_t^k, s_{t+1}^k)\}$.
2. Fit SINDy independently: $\xi_k = \text{SINDy}(X^k, \dot{X}^k)$.
3. Evaluate cross-environment derivative error variance:

$$V_{\text{IRM}} = \text{Var}_k \left[\frac{1}{|D_k|} \sum_{x \in D_k} \|Df_{\text{real}}^k(x) - Df_{\text{sym}}(x)\| \right]. \tag{27}$$

G.2 Formula selection

The IRM-selected formula minimizes cross-environment variance:

$$\xi_{\text{irm}} = \arg \min_{\xi \in \{\xi_1, \ldots, \xi_K\}} V_{\text{IRM}}(\xi). \tag{28}$$

By Theorem 4.20, this yields $\varepsilon_{\text{irm}} \leq \varepsilon_{\text{raw}}$, tightening the OOD bound.

H Hyperparameter Configuration

Table 1: CS-BAPR hyperparameters. Inherited parameters from BAPR are marked with †.

Parameter	**Symbol**	**Default**	**Lean constraint**
SAC core (inherited†)			
Discount factor†	γ	0.99	$0 \leq \gamma < 1$
Soft update rate†	τ	0.005	—
Entropy coefficient†	α	auto-tuned	—
Learning rates†	η	3×10^{-4}	—
Ensemble size†	K	5	—
Batch size†	B	256	—
BAPR inherited†			
Max run length†	H	20	—
Hazard rate†	H_{hazard}	0.02	—
Epistemic weight†	λ_{epi}	0.01	≥ 0
OOD penalty†	β_{ood}	0.1	—
CS-BAPR new			
Symbolic weight	λ_{sym}	0.01	≥ 0
Jacobian weight	μ_{jac}	0.1	—
Gradient clipping	—	1.0	—
SINDy threshold	—	0.1	—
SINDy library degree	—	2	—
NAU regularization	—	0.01	—
IRM penalty weight	—	0.1	—

I Proof Sketch: Component Hypotheses Imply Derivative Bound

Lemma `deriv_error_bound_from_components` (line 157) shows how the three CS-BAPR components combine to satisfy the derivative bound hypothesis of the main theorem:

Lemma I.1. *If the real dynamics f_{real} has constant derivative (smooth physical law), the policy is ε-Jacobian Consistent at the training boundary a, and has L-Arithmetic Inductive Bias, then:*

$$\forall x, \quad |(\pi'(x) - f'_{real}(x))| \leq \varepsilon + L|x - a|. \tag{29}$$

Proof. By triangle inequality:

$$\begin{aligned} |\pi'(x) - f'_{\text{real}}(x)| &= |(\pi'(x) - \pi'(a)) + (\pi'(a) - f'_{\text{real}}(a))| && (30)\\ &\leq |\pi'(x) - \pi'(a)| + |\pi'(a) - f'_{\text{real}}(a)| && (31)\\ &\leq L|x - a| + \varepsilon, && (32) \end{aligned}$$

where the first term uses the Lipschitz derivative property (NAU/NMU) and the second uses Jacobian Consistency at a (SINDy training). The constant-derivative assumption on f_{real} ensures $f'_{\text{real}}(a) = f'_{\text{real}}(x)$. □

This lemma provides the formal bridge from CS-BAPR's three architectural/training assumptions to the hypothesis required by Theorem 4.32.

J Formalization: Data Structure Design

This section documents the Lean 4 data structures used to formalize CS-BAPR, explaining the mathematical rationale behind each design choice.

J.1 NAU Layer

Definition J.1 (NAU Layer, Lean line 799)**.** *An NAU layer of dimension n is a weight vector $w \in \{-1, 0, 1\}^n$ computing $y = \sum_{j=1}^{n} w_j x_j$.*

The discreteness constraint $w_i \in \{-1, 0, 1\}$ is essential: it ensures that the NAU computes an *exact* integer linear combination, making the derivative $\partial y/\partial x_i = w_i$ trivially constant (Theorem 4.9). Without discreteness, the NAU would be a general linear layer with no structural advantage over standard networks. In the Lean formalization, this is encoded as:

```
structure NAULayer (n : ℕ) where
weights : Fin n → ℤ
weight_discrete : ∀ i, weights i = -1 ∨ weights i = 0 ∨ weights i = 1
```

J.2 SINDy Coefficients and Reconstruction

Definition J.2 (SINDy Coefficients, Lean line 470)**.** *Given a basis library $\{\varphi_1, \ldots, \varphi_n\}$, SINDy coefficients are a vector $\xi \in \mathbb{R}^n$ defining the symbolic formula $f_{sym}(x) = \sum_{i=1}^{n} \xi_i \varphi_i(x)$.*

The `reconstruct` function maps coefficients back to the function space, enabling direct comparison with π and f_{real}. The Fréchet derivative of the reconstruction is $Df_{\text{sym}}(x) = \sum_i \xi_i D\varphi_i(x)$, which is used in the Jacobian Consistency definition (Definition 4.6). In Lean:

```
structure SINDyCoeffs (n : ℕ) where
vals : Fin n → ℝ

def SINDyCoeffs.reconstruct (c : SINDyCoeffs n) (lib : BasisLibrary n E F) : E
→ F :=
fun x ⇒ ∑ i, c.vals i • lib.basis i x
```

J.3 IRM Environment

Definition J.3 (Environment, Lean line 715)**.** *An environment consists of a training domain $D_{train} \subseteq E$ and a ground-truth response function $f_{real} : E \to F$.*

IRM filtering (Pillar III) requires comparing symbolic formulas across *multiple* environments. Each environment shares the same state/action spaces but has distinct dynamics parameters (e.g., different masses, friction coefficients). The IRM Consistency predicate (Definition 4.19) requires that a single SINDy coefficient vector ξ achieves low derivative error *simultaneously* across all environments in the list, ensuring causal rather than spurious correlation. In Lean:

```
structure Environment (E F : Type*)
[NormedAddCommGroup E] [NormedSpace ℝ E]
[NormedAddCommGroup F] [NormedSpace ℝ F] where
D_train :  Set E
f_real :  E → F
```

J.4 NMU Unit

Definition J.4 (NMU Unit, Lean line 858)**.** *An NMU unit with coefficient $c \in \mathbb{R}$ computes $f(x) = cx^2$.*

The single-parameter design is deliberate: the NMU's second derivative is $f''(x) = 2c$ (constant), yielding the exact Lipschitz constant $L = 2|c|$ for the first derivative (Theorem 4.10). Richer polynomial units (e.g., $f(x) = c_1x^2 + c_2x^3$) would have *non-constant* second derivatives, making the Lipschitz constant depend on the input domain and losing the global guarantee. In Lean:

```
structure NMUUnit where
coeff :  ℝ
```